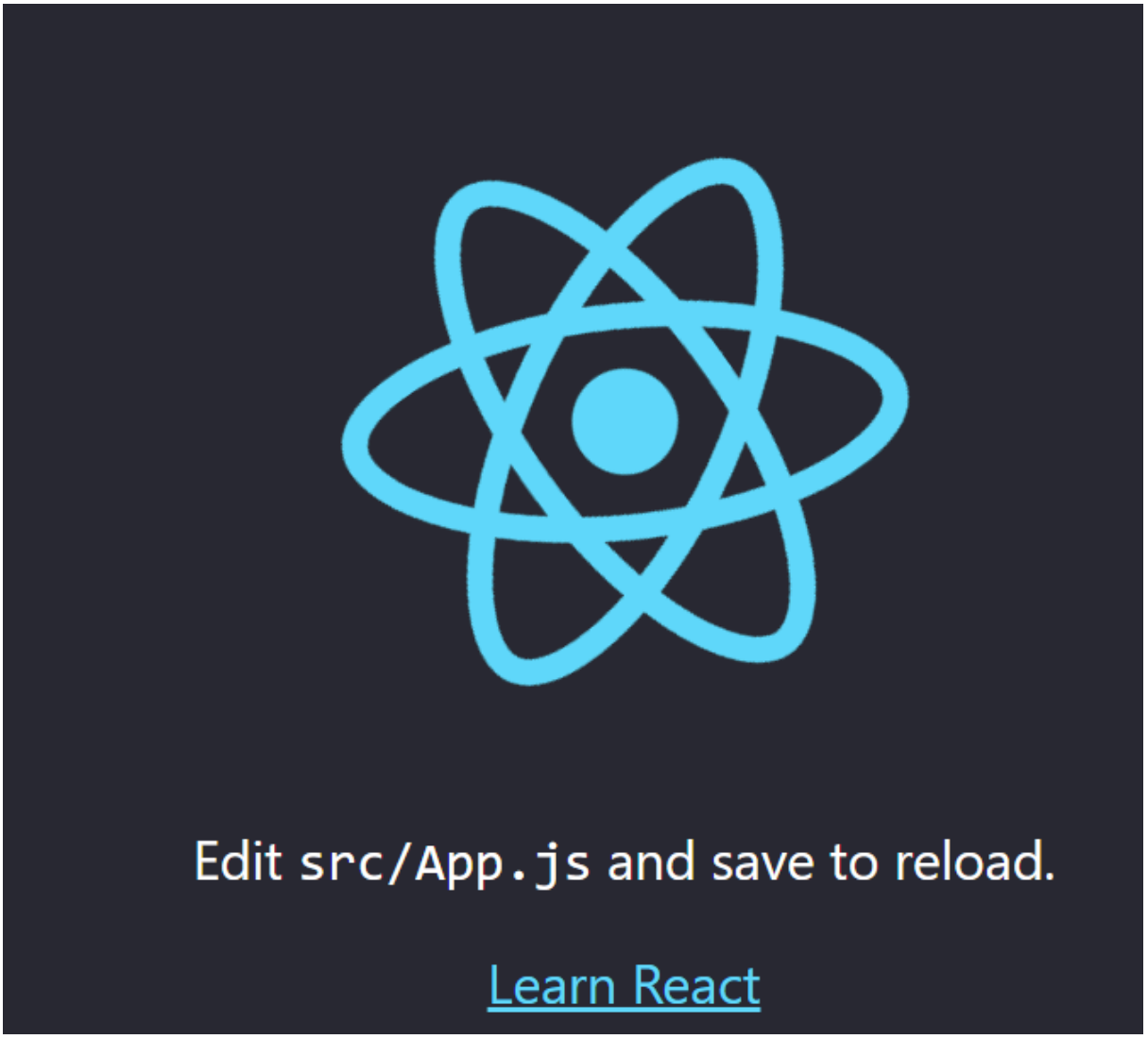
React



Reactを最初に実行するとこんな画面がでる。

```
1  import React from 'react';
2  import ReactDOM from 'react-dom/client';
3  import './index.css';
4  import App from './App';
5  import reportWebVitals from './reportWebVitals';
6
7  const root = ReactDOM.createRoot(document.getElemen
8  root.render(
9    <React.StrictMode>
10     <App />
11   </React.StrictMode>
12 );
13
14 // If you want to start measuring performance in yo
15 // to log results (for example: reportWebVitals(con
16 // or send to an analytics endpoint. Learn more: ht
17 reportWebVitals();
```

これはindex.jsの<App />を表示する。

実際に<App />を消すと何も出ない。

```
import App from './App';
import reportWebVitals fro
```

このAppはApp.jsである。

```
function App() {
    <div className="App">
      <header className="App-header">
        <img src={logo} className="App-logo" alt="logo'
        <p>
          Edit <code>src/App.js</code> and save to rel
        </p>
        <a
          className="App-link"
          href="https://reactjs.org"
          target="_blank"
          rel="noopener noreferrer"
        >
          Learn React
        </a>
      </header>
    </div>
  );
}

export default App;
```

```
<div className="App">
  {/* <header className=
    <img src={logo} cla
    <p>
      Edit <code>src/Ap
    </p>
    <a
      className="App-li
      href="https://rea
      target="_blank"
      rel="noopener nor
    >
      Learn React
    </a>
  </header> */}
  Hello React
```

このようにHello Reactを入力すると

Hello React

こんな画面に変わる。

```
PS C:\Users\yun\Desktop\React\my-app> npm run build

The build folder is ready to be deployed.
You may serve it with a static server:

  npm install -g serve
  serve -s build

Find out more about deployment here:

  https://cra.link/deployment
```

buildは　npm run buildをしてする。

```
PS C:\Users\yun\Desktop\React\my-app> npx serve -s build
Need to install the following packages:
serve@14.2.6
Ok to proceed? (y) y

   Serving!

   - Local:    http://localhost:3000
```

そして、 npx serve -s buildを入力すればserverが開く。

コンポーネント

Reactの本質はカスタムタグを作る技術である。

```
return (
  <div>
    <header>
      <h1><a href="/">WEB</a></h1>
    </header>
    <nav>
      <ol>
        <li><a href="/read/1">html</a></li>
        <li><a href="/read/2">css</a></li>
        <li><a href="/read/3">js</a></li>
      </ol>
    </nav>
    <article>
      <h2>Welcome</h2>
      Hello, WEB
    </article>
  </div>
```

このコードはheader, nav, articleになる。

WEB

1. html
2. css
3. js

Welcome

Hello, WEB

実行するとこうなる。

```
function Header() {
  return (
    <header>
      <h1><a href="/">WEB</a></h1>
    </header>

  )
}

function App() {
  return (
    <div>
      <Header></Header>
      <nav>
```

そして、カスタムタグのためにHeaderの関数を作ってAppに追加する。

```
WARNING in [eslint]
```

すると、eslintというエラーが出た。

```
<div>
  <header>
    <h1><a href="/">WEB</a></h1>
  </header>
</div>
```

headerにdivをつけてかいけつした。

```
function Header() {
  return (
    <div>
      <header>
        <h1><a href="/">Hello</a></h1>
      </header>
    </div>
  )
}

function App() {
  return (
    <div>
      <Header></Header>
      <Header></Header>
      <Header></Header>
      <Header></Header>
```

今回はHelloに変えて4回入力した。

Hello

Hello

Hello

Hello

1. html
2. css
3. js

Welcome

Hello, WEB

このように効率的な使用のため、作ったカスタムタグをReactにはコンポーネントという。